

A 郑东图书馆的搬书日 (Book Moving Day)

- 时间限制: 1.0 s
- 内存限制: 256 MB

题目背景

北洋园校区的郑东图书馆（各校区公认的“最美图书馆”）最近购进了一批珍贵的新书。

作为智能与计算学部的新生志愿者，你们接到了一个任务：将这批书从一楼大厅搬运到四楼的阅览室。

题目描述

共有 N 本书排成一排，第 i 本书的重量为 A_i 。

由于这些书是按照特定顺序分类好的，搬运时不能改变书的排列顺序。也就是说，每个人只能搬运连续的一段书。

现在有 M 位志愿者参与搬书。为了尽快完成任务，需要将这 N 本书分成 M 个连续的非空片段，每位志愿者负责搬运其中一个片段。

由于每位志愿者的体力是有限的，为了公平起见，我们希望**体力负担最重的那位志愿者，搬运的重量尽可能小**。

请你规划一种分配方案，使得**所有志愿者中，搬运重量最大的那个人，其搬运的重量最小**。请输出这个最小的“最大重量”。

输入格式

第一行包含两个正整数 N 和 M ，分别表示书的数量和志愿者的数量。

第二行包含 N 个正整数 $A_1, A_2, \dots, A_N$ ，表示每本书的重量。

输出格式

输出一个整数，表示在最优分配方案下，搬运重量最大的志愿者所搬运的重量。

数据范围

- 对于 30% 的数据： $1 \leq M \leq N \leq 100$
- 对于 100% 的数据： $1 \leq M \leq N \leq 10^6, 1 \leq A_i \leq 10^4$

样例输入 1

```
5 3
4 2 4 5 1
```

样例输出 1

```
6
```

样例解释 1

我们需要将序列 `[4, 2, 4, 5, 1]` 分成 3 段。

- 方案 1: `[4, 2]`, `[4]`, `[5, 1]` -> 和分别为 6, 4, 6。最大值为 6。
- 方案 2: `[4]`, `[2, 4]`, `[5, 1]` -> 和分别为 4, 6, 6。最大值为 6。
- 方案 3: `[4, 2, 4]`, `[5]`, `[1]` -> 和分别为 10, 5, 1。最大值为 10。

...

在所有可能的切分方案中，最大值的最小可能结果是 6。

样例输入 2

```
5 2
10 10 10 10 10
```

样例输出 2

```
30
```

样例解释 2

分成 2 段，最优分法是 `[10, 10]` 和 `[10, 10, 10]`（或者反过来），和分别为 20 和 30。最大值是 30。

题目解析 (For Teacher/Setter)

考察点

1. **二分答案 (Binary Search on Answer)**: 答案具有单调性。如果限制每个人最多搬 X 重量的书能完成任务，那么限制每个人最多搬 $X + 1$ 重量肯定也能完成。
2. **贪心策略 (Greedy)**: 在验证某个答案 X 是否可行时，需要尽可能“贪心”地往一个人的手里塞书，直到塞不下为止，再换下一个人。

解题思路

1. 确定搜索范围：

我们需要寻找的是一个重量上限 W 。

- **下界 (L)**: 至少要能搬动最重的那本书，即 $\max(A_i)$ 。如果限制小于单本书的重量，那本书就没人能搬了。
- **上界 (R)**: 最坏情况是一个人搬所有的书，即 $\sum A_i$ 。
- 二分过程：

取中间值 $Mid = (L + R) / 2$ 。

我们需要编写一个 `check(limit)` 函数，判断：如果要求每个人搬运的重量都不超过 `limit`，能否在 M 个人以内搬完？

3. `check` 函数的逻辑 (贪心) :

- 初始化需要的志愿者人数 `count = 1`，当前志愿者已搬运重量 `current_sum = 0`。

- 从第 1 本书遍历到第 N 本书：
 - 如果 `current_sum + A[i] <= limit`：这就意味着当前志愿者还能搬得动这本书，直接加上。
 - 否则 (`current_sum + A[i] > limit`)：当前志愿者搬不动了，需要换一个新的志愿者。
 - `count++` (人数加1)
 - `current_sum = A[i]` (新的人从这本书开始搬)
- 遍历结束后，如果需要的总人数 `count <= M`，说明这个限制 `limit` 是可行的（甚至可能太宽松了），我们尝试减小答案，令 $R = Mid - 1$ 。
- 如果 `count > M`，说明限制太严了，人不够用，需要增大限制，令 $L = Mid + 1$ 。

4. 复杂度：

- `check` 函数复杂度为 $O(N)$ 。
- 二分搜索的次数为 $O(\log(\sum A_i))$ 。
- 总复杂度 $O(N \log(\sum A_i))$ ，对于 $N = 10^5$ 完全可过。

标准代码 (C++)

```
#include <iostream>
#include <vector>
#include <numeric> // 用于 accumulate
#include <algorithm> // 用于 max_element

using namespace std;

// 检查函数：判断如果最大重量限制为 limit，是否能在 m 个人内搬完
bool check(long long limit, int n, int m, const vector<int>& a) {
    int count = 1; // 需要的志愿者数量，至少需要1个人
    long long current_sum = 0;

    for (int i = 0; i < n; ++i) {
        // 如果单本书的重量都超过了限制，那肯定不行（虽然在二分下界处理了，加个判断更稳）
        if (a[i] > limit) return false;

        if (current_sum + a[i] <= limit) {
            // 当前志愿者还能搬
            current_sum += a[i];
        } else {
            // 当前志愿者搬不动了，换下一个人
            count++;
            current_sum = a[i];
        }
    }

    // 如果需要的人数小于等于 m，说明这个 limit 是可行的
    return count <= m;
}

int main() {
    // 优化输入输出效率
```

```

ios_base::sync_with_stdio(false);
cin.tie(NULL);

int n, m;
if (!(cin >> n >> m)) return 0;

vector<int> a(n);
long long sum = 0;
int max_val = 0;

for (int i = 0; i < n; ++i) {
    cin >> a[i];
    sum += a[i];
    if (a[i] > max_val) max_val = a[i];
}

// 二分答案
// 左边界: 最大的一本书的重量 (如果小于这个值, 那本书就没人能搬)
// 右边界: 所有书的总重量 (一个人全搬)
long long left = max_val;
long long right = sum;
long long ans = right;

while (left <= right) {
    long long mid = left + (right - left) / 2;

    if (check(mid, n, m, a)) {
        ans = mid;    // 记录可行解
        right = mid - 1; // 尝试更小的最大值
    } else {
        left = mid + 1; // 限制太小了, 需要增大
    }
}

cout << ans << endl;

return 0;
}

```

B 二元运算

解法一: 线段树

定义 $\text{rol}_{32}(y, k)$ 的逆运算循环右移 $\text{ror}_{32}(y, k) = (y \gg k) \mid (y \ll (32 - k))$

我们可以发现, 对于两个连续的函数:

- $f_1(x) = \text{rol}_{32}(a_1 \oplus x, b_1)$
- $f_2(x) = \text{rol}_{32}(a_2 \oplus x, b_2)$

它们的组合 $f_2(f_1(x))$ 也是一个相同形式的函数:

$$f_2(f_1(x)) = \text{rol}_{32}(a_{\text{new}} \oplus x, b_{\text{new}})$$

其中（推导见附录）：

- $a_{\text{new}} = a_2 \oplus \text{ror}_{32}(a_1, b_2)$
- $b_{\text{new}} = (b_1 + b_2) \bmod 32$

开一棵线段树，每个节点存储一个函数参数 (a, b) ，表示该区间内所有函数的组合结果。然后套用单点修改区间查询的板子即可。

时间复杂度： $O(n \log n)$ （不过线段树常数很大）。

解法二：分块

另一种思路是使用分块算法。我们将序列分成 $\sqrt{n}$ 大小的块，对每个完整的块预处理其函数组合结果。

- 对每个块，计算整个块的函数组合结果 (a, b) 。
- 当修改某个元素时，只需要重新计算所在块的预处理结果。
- 对于查询区间 $[l, r]$ ：
 1. 如果区间在一个块内，直接暴力计算
 2. 否则，分别处理左右两端不完整的块（暴力计算），中间完整的块使用预处理结果

时间复杂度： $O(n\sqrt{n})$ （常数比较小，但需要注意细节）。

附录

题目来源

这道题来自 [D-【模板】区间一次函数复合求值 牛客挑战赛55](#)。

考虑到学校 OJ 的性能，降低了数据量，这使得线段树解法的时间复杂度优势更不明显了。事实上，在这个数据量下分块只比线段树慢 10% 左右。

将原题的 $f(x) = ax + b$ 改成了 $f(x) = \text{rol}_{32}(a \oplus x, b)$ ，略微提升了合并表达式的推导难度。

合并表达式推导

$$\begin{aligned} f_2(f_1(x)) &= \text{rol}_{32}(a_2 \oplus \text{rol}_{32}(a_1 \oplus x, b_1), b_2) \\ &= \text{rol}_{32}(a_2 \oplus \text{rol}_{32}(a_1, b_1) \oplus \text{rol}_{32}(x, b_1), b_2) \\ &= \text{rol}_{32}(\text{rol}_{32}(\text{ror}_{32}(a_2, b_1), b_1) \oplus \text{rol}_{32}(a_1, b_1) \oplus \text{rol}_{32}(x, b_1), b_2) \\ &= \text{rol}_{32}(\text{rol}_{32}(\text{ror}_{32}(a_2, b_1) \oplus a_1 \oplus x, b_1), b_2) \\ &= \text{rol}_{32}(\text{ror}_{32}(a_2, b_1) \oplus a_1 \oplus x, b_1 + b_2) \\ &= \text{rol}_{32}(a_{\text{new}} \oplus x, b_{\text{new}}) \end{aligned}$$

分块代码：

```
#include <bits/stdc++.h>
using namespace std;

uint32_t rol(uint32_t x, uint32_t k) { return (x << k) | (x >> (32 - k)); }
uint32_t ror(uint32_t x, uint32_t k) { return (x >> k) | (x << (32 - k)); }

const int maxn = 3e5 + 10;
const int BSIZE = 500; // 块大小约为sqrt(3e5)≈547

int n, m;
```

```

uint32_t A[maxn];
uint32_t B[maxn];

// 预处理每一块的信息
struct BlockInfo {
    uint32_t a, b;
};

BlockInfo block_info[maxn / BSIZE + 10]; // 存储每个块的合并信息
int block_cnt;                          // 块的数量

// 更新指定块的信息
void update_block(int block_id) {
    int start = block_id * BSIZE + 1;
    int end = min(start + BSIZE - 1, n);

    // 计算该块的合并信息
    if (start > end)
        return;

    // 从右到左计算函数组合
    uint32_t res_a = A[end];
    uint32_t res_b = B[end];

    for (int i = end - 1; i >= start; i--) {
        // 直接从左到右计算
        uint32_t new_a = A[i] ^ ror(res_a, B[i]);
        uint32_t new_b = (B[i] + res_b) % 32;
        res_a = new_a;
        res_b = new_b;
    }

    block_info[block_id].a = res_a;
    block_info[block_id].b = res_b;
}

// 更新单个元素后，更新对应的块信息
void update_element(int pos, uint32_t a, uint32_t b) {
    A[pos] = a;
    B[pos] = b;
    int block_id = (pos - 1) / BSIZE;
    update_block(block_id);
}

// 直接计算区间[l,r]的结果（暴力）
uint32_t direct_query(int l, int r, uint32_t x) {
    uint32_t res = x;
    for (int i = l; i <= r; i++) {
        res = rol(A[i] ^ res, B[i]);
    }
    return res;
}

// 查询区间[l,r]的结果
uint32_t query(int l, int r, uint32_t x) {
    int l_block = (l - 1) / BSIZE;

```

```

int r_block = (r - 1) / BSIZE;

if (l_block == r_block) {
    // 在同一块内，直接计算
    return direct_query(l, r, x);
} else {
    uint32_t res = x;

    // 处理左侧不完整的块
    int l_end = (l_block + 1) * BSIZE;
    res = direct_query(l, l_end, res);

    // 处理中间完整的块
    for (int i = l_block + 1; i < r_block; i++) {
        // 使用预处理的块信息
        res = rol(block_info[i].a ^ res, block_info[i].b);
    }

    // 处理右侧不完整的块
    int r_start = r_block * BSIZE + 1;
    res = direct_query(r_start, r, res);

    return res;
}
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);

    cin >> n >> m;
    for (int i = 1; i <= n; i++) {
        cin >> A[i];
    }
    for (int i = 1; i <= n; i++) {
        cin >> B[i];
    }

    // 初始化块信息
    block_cnt = (n - 1) / BSIZE + 1;

    for (int i = 0; i < block_cnt; i++) {
        update_block(i);
    }

    for (int i = 1; i <= m; i++) {
        int op;
        cin >> op;

        if (op == 1) {
            int pos;
            uint32_t a, b;
            cin >> pos >> a >> b;
            update_element(pos, a, b);
        } else {

```

```

        int l, r;
        uint32_t x;
        cin >> l >> r >> x;
        uint32_t ans = query(l, r, x);
        cout << ans << '\n';
    }
}

return 0;
}

```

线段树代码：

```

#include <bits/stdc++.h>
using namespace std;

uint32_t rol(uint32_t x, uint32_t k) { return (x << k) | (x >> (32 - k)); }
uint32_t ror(uint32_t x, uint32_t k) { return (x >> k) | (x << (32 - k)); }

struct Node {
    uint32_t a;
    uint32_t b;

    Node(void) = default;
    Node(uint32_t _a, uint32_t _b) : a(_a), b(_b) {}

    Node(const Node &l, const Node &r) {
        a = l.a ^ ror(r.a, l.b);
        b = (l.b + r.b) % 32;
    }
};

const int maxn = 3e5 + 10;

int n, m;
uint32_t A[maxn];
uint32_t B[maxn];
Node tree[maxn << 2];

#define mid ((l + r) / 2)
#define ls (k << 1)
#define rs (k << 1 | 1)

void build(int k, int l, int r) {
    if (l == r) {
        tree[k] = Node(A[l], B[l]);
        return;
    }

    build(ls, l, mid);
    build(rs, mid + 1, r);

    tree[k] = Node(tree[ls], tree[rs]);
}

void build(void) { build(1, 1, n); }

```



```

void update(int k, int l, int r, int pos, const Node &val) {
    if (l == r) {
        tree[k] = val;
        return;
    }

    if (pos <= mid) {
        update(ls, l, mid, pos, val);
    } else {
        update(rs, mid + 1, r, pos, val);
    }
    tree[k] = Node(tree[ls], tree[rs]);
}

void update(int pos, const Node &val) { update(1, 1, n, pos, val); }

Node query(int k, int l, int r, int ql, int qr) {
    if (ql <= l && r <= qr) {
        return tree[k];
    }

    if (qr <= mid) {
        return query(ls, l, mid, ql, qr);
    } else if (ql > mid) {
        return query(rs, mid + 1, r, ql, qr);
    } else {
        return Node(query(ls, l, mid, ql, qr), query(rs, mid + 1, r, ql, qr));
    }
}

Node query(int ql, int qr) { return query(1, 1, n, ql, qr); }

#undef mid
#undef ls
#undef rs

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);

    // auto start = chrono::high_resolution_clock::now();

    cin >> n >> m;
    for (int i = 1; i <= n; i++) {
        cin >> A[i];
    }
    for (int i = 1; i <= n; i++) {
        cin >> B[i];
    }

    build();

    for (int i = 1; i <= m; i++) {
        int op;
        cin >> op;

```

```

        if (op == 1) {
            int pos;
            uint32_t a, b;
            cin >> pos >> a >> b;
            update(pos, Node(a, b));
        } else {
            int l, r;
            uint32_t x;
            cin >> l >> r >> x;
            Node node = query(l, r);
            cout << rol(node.a ^ x, node.b) << '\n';
        }
    }

    // auto end = chrono::high_resolution_clock::now();
    // cerr << chrono::duration_cast<chrono::milliseconds>(end - start).count()
    << "ms" << endl;

    return 0;
}

```

C 北洋园的位运算挑战

一、解题思路

1. 问题重述与关键观察

我们需要将 n 个非负整数分配到两个非空集合 S_1 和 S_2 中，使得 $\text{AND}(S_1) - \text{AND}(S_2)$ 最大化。这里 $\text{AND}(S)$ 表示集合 S 中所有元素的按位与结果。

核心观察：存在一个最优划分，其中一个集合只包含一个元素。

2. 引理证明：单元素集合的最优性

引理：对于任意最优划分 (S, T) ，如果 $|S| \geq 2$ 且 $\text{AND}(S) \geq \text{AND}(T)$ ，那么存在一个元素 $x \in S$ ，将 x 从 S 移动到 T 后，新的划分 $(S \setminus \{x\}, T \cup \{x\})$ 的差值不会减小。

证明：

设当前划分 (S, T) 满足 $\text{AND}(S) \geq \text{AND}(T)$ ，记：

- $B = \text{AND}(S \setminus \{x\})$ ，其中 $x \in S$
- $C = \text{AND}(T)$

那么：

- $\text{AND}(S) = B \& x$
- $\text{AND}(T) = C$

当前差值： $d = (B \& x) - C$

现在将 x 从 S 移动到 T ：

- 新的 $S' = S \setminus \{x\}$ ， $\text{AND}(S') = B$
- 新的 $T' = T \cup \{x\}$ ， $\text{AND}(T') = C \& x$

新的差值: $d' = B - (C \& x)$

计算差值变化:

$$d' - d = [B - (C \& x)] - [(B \& x) - C]$$

$$= B - C \& x - B \& x + C$$

$$= (B - B \& x) + (C - C \& x)$$

由于按位与运算的性质:

- $B \& x \leq B$, 所以 $B - B \& x \geq 0$
- $C \& x \leq C$, 所以 $C - C \& x \geq 0$

因此 $d' - d \geq 0$, 即移动 x 后差值不会减小。

推论: 重复上述过程, 我们可以不断将 S 中的元素移动到 T 中, 直到 S 中只剩下一个元素。每次移动都不会使差值减小, 因此最终得到的划分 (其中一个集合只包含一个元素) 至少与原最优划分一样优。所以, 存在一个最优划分, 其中一个集合只包含一个元素。

3. 算法设计思路

基于引理, 我们可以假设最优划分中, 一个集合只包含单个元素 $A[i]$, 另一个集合包含其余所有元素。那么问题转化为:

对于每个 $i \in [0, n-1]$, 计算:

$$\text{差值}(i) = A[i] - \text{AND}(\text{所有其他元素})$$

然后取所有差值 (i) 的最大值。

4. 高效计算其余元素的AND值

直接对每个 i 计算其余元素的AND值需要 $O(n^2)$ 时间, 不可接受。我们需要优化。

技巧: 使用前缀与和后缀与数组。

定义:

- $\text{prefix}[i] = A[0] \& A[1] \& \dots \& A[i]$ (前 $i+1$ 个元素的AND)
- $\text{suffix}[i] = A[i] \& A[i+1] \& \dots \& A[n-1]$ (从 i 到末尾元素的AND)

那么, 对于任意 i , 其余元素的AND值可以通过以下方式计算:

$$\text{rest_and}(i) = \text{prefix}[i-1] \& \text{suffix}[i+1] \text{ (当 } 0 < i < n-1 \text{ 时)}$$

边界情况:

- $i = 0$ 时: $\text{rest_and}(0) = \text{suffix}[1]$
- $i = n-1$ 时: $\text{rest_and}(n-1) = \text{prefix}[n-2]$

5. 算法步骤

1. **输入处理:** 读取整数 n 和数组 A

2. **预处理前缀与数组:**

- $\text{prefix}[0] = A[0]$
- 对于 i 从 1 到 $n-1$: $\text{prefix}[i] = \text{prefix}[i-1] \& A[i]$

3. **预处理后缀与数组:**

- $\text{suffix}[n-1] = A[n-1]$

- 对于i从n-2到0: $\text{suffix}[i] = \text{suffix}[i+1] \& A[i]$

4. 枚举每个元素作为单元元素集合:

- 对于i从0到n-1:
 - 计算rest_and
 - 计算差值 = $A[i] - \text{rest_and}$
 - 更新最大差值

5. 输出最大差值

6. 正确性证明补充

我们已经证明了存在单元元素集合的最优划分。接下来需要证明通过枚举每个元素作为单元元素集合，并计算与其余元素AND值的差值，能够找到最优解。

设最优划分为 (S, T) ，其中 S^* 只包含一个元素 $A[k]$ 。那么：

- $\text{AND}(S^*) = A[k]$
- $\text{AND}(T^*) = \text{AND}(\text{所有其他元素}) = \text{rest_and}(k)$

因此，差值 = $A[k] - \text{rest_and}(k)$ 。我们的算法计算了每个i对应的 $A[i] - \text{rest_and}(i)$ ，并取最大值，所以一定能找到这个最优的k。

7. 时间复杂度分析

- 计算前缀数组: $O(n)$
- 计算后缀数组: $O(n)$
- 枚举每个元素并计算差值: $O(n)$
- 总时间复杂度: $O(n)$
- 空间复杂度: $O(n)$ (存储前缀和后缀数组)

8. 实现细节注意事项

1. **位运算优先级**: 注意按位与运算符 $\&$ 的优先级低于比较运算符，表达式中需要适当加括号。
2. **输入输出优化**: 对于n最大 10^6 的情况，建议使用快速输入输出。

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

int main()
{
    // 优化输入输出
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    vector<int> a(n);
    for (int i = 0; i < n; ++i)
```

```

{
    cin >> a[i];
}

// 前缀与数组
vector<int> prefix(n);
prefix[0] = a[0];
for (int i = 1; i < n; ++i)
{
    prefix[i] = prefix[i - 1] & a[i];
}

// 后缀与数组
vector<int> suffix(n);
suffix[n - 1] = a[n - 1];
for (int i = n - 2; i >= 0; --i)
{
    suffix[i] = suffix[i + 1] & a[i];
}

int max_diff = 0;

for (int i = 0; i < n; ++i)
{
    int rest_and;

    if (i == 0)
    {
        rest_and = suffix[1];
    }
    else if (i == n - 1)
    {
        rest_and = prefix[n - 2];
    }
    else
    {
        rest_and = prefix[i - 1] & suffix[i + 1];
    }

    int diff = a[i] - rest_and;
    if (diff > max_diff)
    {
        max_diff = diff;
    }
}

cout << max_diff << endl;

return 0;
}

```

D 若叶睦的黄瓜田

题意

现有一个长度为 n 的数组 a ，可以进行任意次操作（包括不操作），每次操作可以选择标号 $2 \sim n - 1$ 进行操作，操作规则如下：选择标号 i ，将数组 a_i 减2，数组 a_{i-1} 减1，数组 a_{i+1} 减1。问最后是否能让数组变成全为0。

输入

第一行包含一个整数 t ($1 \leq t \leq 10^4$) — 测试用例的数量。

每个用例的第一行包含一个整数 n ($3 \leq n \leq 2 \cdot 10^5$) — 数组的数量。

每个用例的第二行包含 n 个整数 $a_1, a_2, \dots, a_n$ ($0 \leq a_j \leq 10^9$) — 数组的初始值。

输出

对于每个测试用例，如果可以让所有数组值恰好为零（不出现负数），输出 yes；否则，输出 no。

题解

能帮助理解本题的一个细节：操作顺序不会改变操作结果，如果不得不做出某个操作，那么不妨优先完成这个操作。

我们不妨从数组的第一个元素开始考虑。

如果 a_1 不为0的话，想要把 a_1 减到0，**唯一**的方式是对 a_2 进行操作，操作次数就是 a_1 的大小，如果操作导致 a_2 或 a_3 减成负数，就说明无法实现目标。

同理，如果 a_1 原本就是0或者经过操作后变成0，就对 a_2 做同样的考虑，此时不能再对 a_2 为中心进行任何操作，否则会使得 a_1 减为负数，若 a_2 仍为正数，只能对 a_3 进行操作，操作次数是当前 a_2 的值，把 a_2 减到0，再检查 a_3 和 a_4 ，如果减到负数，说明无法实现目标。

于是依次操作，直到 a_{n-2} ，当 a_{n-2} 减为0时， a_{n-1} 和 a_n 必须也为0，因为已经无法再做更多操作了，代码实现时需要特别注意判断。

```
#include<bits/stdc++.h>
using namespace std;
int a[200005];
int main()
{
    int t;
    cin>>t;
    while(t-->0)
    {
        int n;
        cin>>n;
        for(int i=1;i<=n;i++)
        {
            cin>>a[i];
        }
        int f=1;
        for(int i=1;i<=n-2;i++)
        {
            a[i+1]-=a[i]*2;
            a[i+2]-=a[i];
        }
    }
}
```

```

        a[i]=0;
        if(a[i+1]<0||a[i+2]<0)
        {
            f=0;
            break;
        }
    }
    if(a[n-1]!=0||a[n]!=0) f=0;
    if(f==1) cout<<"yes\n";
    else cout<<"no\n";
}
}

```

E 字符串

题意

现有一个由 N 个小写字母组成的字符串，你可以在后面继续添加任意个小写字母（包括0个），问最少需要添加多少个字母使字符串成为一个回文串。

回文串的定义：一个字符串如果从左往右读和从右往左读完全相同，则称为回文串。例如，"aba"、"racecar"、"a"都是回文串；而"ab"、"abc"不是回文串。

输入

第一行包含一个整数 N ($1 \leq N \leq 1 \times 10^6$)，代表字符串的长度

第二行包含一个由 N 个小写字母（从 a 到 z）组成的字符串。

输出

输出一个整数，代表使给定小写字母字符串成为回文串需在字符串末尾加入字母的最少数量。

题解

假如输入的字符串为 s (下标从1开始)，最后的答案为 k ，则最后的字符串 s 的长度为 $N + k$ 且满足 $s[i] == s[N + k + 1 - i]$ ，对于 $1 \leq i \leq N + k$ 都满足。

由于最后 k 个字符由我们随意添加，所以可以保证该部分回文性质满足，我们只需要保证 $s[k + 1, N]$ 是一个回文串即可。所以这道题是需要找到最长回文后缀（长度为 len ），答案为 $N - len$ 。

我们可以通过哈希，KMP，manacher的方法来解决该问题，时间复杂度都是 $O(N)$

```

//哈希
#include<bits/stdc++.h>
using namespace std;
const int maxn = 1e6 + 10;
const long long mod = 1e9 + 7, base = 13331;
long long n, pw[maxn], hs1[maxn], hs2[maxn];
void init_pw() {
    pw[0] = 1;
    for (int i = 1; i <= n; i++) {
        pw[i] = pw[i - 1] * base % mod;
    }
}
void init_hs(string str, long long hs[]) {

```

```

hs[0] = 1;
for (int i = 1; i <= n; i++) {
    hs[i] = (hs[i - 1] * base + str[i] - 'a') % mod;
}
}

int get_hs(int l, int r, long long hs[]) {
    int res = (hs[r] - hs[l - 1] * pw[r - l + 1]) % mod;
    return (res + mod) % mod;
}

int main(){
    string s, ns;
    cin >> n >> s;
    ns = s; reverse(ns.begin(), ns.end());
    s = ' ' + s;
    ns = ' ' + ns;
    init_pw();
    init_hs(s, hs1);
    init_hs(ns, hs2);
    long long ans = 0;
    for (int i = n / 2 + 1; i <= n; i++) {
        //检查以i为回文中心能否构成回文后缀
        int l1 = 2 * i - n, r1 = i;
        int l2 = i, r2 = n;
        if (get_hs(l1, r1, hs1) == get_hs(n + 1 - r2, n + 1 - l2, hs2)) {
            ans = max(ans, 2 * (n - i) + 1);
            break;
        }
    }

    //检查以i,i + 1为回文中心能否构成回文后缀
    for (int i = (n + 1) / 2; i <= n; i++) {
        int l1 = 2 * i + 1 - n, r1 = i;
        int l2 = i + 1, r2 = n;
        if (get_hs(l1, r1, hs1) == get_hs(n + 1 - r2, n + 1 - l2, hs2)) {
            ans = max(ans, 2 * (n - i));
            break;
        }
    }

    cout << n - ans << endl;
}

```

```

//KMP
#include<bits/stdc++.h>
using namespace std;
const int maxn = 2e6 + 10;
long long n, nxt[maxn];
void get_nxt(string str) {
    nxt[1] = 0;
    int j = 1;
    for (int i = 2; i < str.length(); i++) {
        while (j > 1 && str[j] != str[i]) j = nxt[j - 1] + 1;
        if (str[j] == str[i]) j++;
        nxt[i] = j - 1;
    }
}

int main(){

```



```

string s, ns;
cin >> n >> s;
ns = s; reverse(ns.begin(), ns.end());
s = ' ' + ns + '#' + s; ///#作为分隔符,构造新的s以匹配nxt数组含义
get_nxt(s);
cout << n - nxt[s.length() - 1] << endl;
}

```

```

//manacher
#include<bits/stdc++.h>
using namespace std;
const int maxn = 2e6 + 10;
int ans[maxn];
void manacher(string str) {
    int len = str.length();
    int r = -1, l = 0;
    for(int i = 0; i < len; i++) {
        int now = (i > r) ? 0 : min(ans[l + r - i], r - i);
        while (i - now - 1 >= 0 && i + now + 1 <= len && str[i - now - 1] ==
str[i + now + 1]) now++;
        ans[i] = now;
        if (i + now > r) {
            r = i + now; l = i - now;
        }
    }
}
int main(){
    int n;
    string s, ns;
    cin >> n >> ns;
    s = '#';
    for (int i = 1; i <= n; i++) {
        s += ns[i - 1];
        s += '#';
    }
    manacher(s);
    for (int i = 1; i <= 2 * n; i++) { ///逐个检验回文中心,看是不是后缀
        if (i + ans[i] == 2 * n) {
            if (i % 2) {
                cout << n - (2 * n - i - 1) - 1 << endl;
            } else cout << n - (2 * n - i) << endl;
            break;
        }
    }
}
}

```

F 子串数目

题意

这里将题意简化,就是m个1, n-m个0,组成的字符串中,含有1的子串数目最多是多少

题解

一个长度固定为 n 的字符串的子串数目是确定的，就是 $n * (n + 1) / 2$

长度为1的子串有 n 个，长度为2的子串有 $n-1$ 个，...，长度为 n 的子串为一个，因此所有子串数目是一个等差数列求和， $1 + 2 + \dots + n = n * (n + 1) / 2$

因此我们再次转化题意，总数目确定，只需要减去只含0的子串数目，就是含有1的子串数目。

我们需要想办法最小化只含0的子串数目。

由于题目中0的数目是确定的，连续的0会使得只含0的子串数目变多，例如一个字符串中，连续0的数目分别为 $a_1, a_2, \dots, a_k$ ，其中 $a_1 + a_2 + \dots + a_k = n - m$ ，只含0的子串数目就是

$a_1 * (a_1 + 1) / 2 + a_2 * (a_2 + 1) / 2 + \dots + a_k * (a_k + 1) / 2$ ，因此， k 越大，且把 $n-m$ 平均分给这 k 个 a 是最优的。

我们需要用1把0隔开，最多可以使得 k 等于 $m+1$ 。

平均分可以先让 $(n - m) / (m + 1)$ ，余数 p 作为多出来的部分，选择其中 p 个加1即可。

代码如下：

```
#include<bits/stdc++.h>
#define int long long
using namespace std;
const int mod=1000000007;
const int inf=1e18+10;
const double eps=1e-6;
signed main(){
    ios::sync_with_stdio(false);
    cin.tie(0);cout.tie(0);
    int t;
    cin>>t;
    while(t--){
        int n,m;
        int ans=0;
        cin>>n>>m;
        int tem=(n-m)/(m+1);
        int da=n-m-tem*(m+1);
        int xiao=m+1-da;
        ans=da*(tem+2)*(tem+1)/2;
        ans+=xiao*(tem)*(tem+1)/2;
        cout<<n*(n+1)/2-ans<<'\n';
    }
    return 0;
}
```

G 博士的无理改造

题意

在东云研究所，天才但随性的 博士 给 机器人东云名乃 进行了一系列的"大改造"。

名乃背上的发条决定了她的最大瞬间输出功率，记为 W (瓦特)。

一旦瞬间消耗的功率超过 W ，名乃就会过热宕机，甚至引发以研究所为中心的大爆炸。

虽然博士是个天才，但她完全没有计划性。她给名乃安装了 N 个奇怪的功能（比如"额头瑞士卷出货机"、"手必杀光线"、"脚底喷气机"等），并制定了一张混乱的时间表来测试这些功能。

第 i 个功能的测试计划包含三个参数：

- 开启时间 S_i ：博士开启该功能的时刻。
- 关闭时间 T_i ：博士关闭该功能的时刻。注意：实际运行时间为左闭右开区间 $[S_i, T_i)$ ，即功能在 T_i 这一刻刚好停止，不消耗功率。
- 功率消耗 P_i ：该功能运行期间，每分钟持续占用的功率。

所有开启的功能都是并发运行的。这意味着在任意时刻 t ，名乃当前的总功率消耗是所有正在该时刻运行的功能的 P_i 之和。

(^_^) 坂本先生感觉非常不妙，于是他来求助于你：请帮他计算一下，按照博士的这张测试时间表，在测试期间的任意时刻，名乃的总功率消耗是否都能保持在安全范围（小于等于 W ）内？

输入

第一行包含整数 N （功能数量）和 W （名乃的最大功率）。接下来 N 行，每行三个整数 S_i, T_i, P_i 。按照如下格式进行输入

```
N W
S1 T1 P1
⋮
SN TN PN
```

- $1 \leq N \leq 2 \times 10^5$
- $0 \leq S_i < T_i \leq 2 \times 10^5$
- $1 \leq P_i \leq 10^9$
- $1 \leq W \leq 10^{18}$
- 所有输入都是整数。

输出

如果名乃能安全运行，输出 "Yes"。

如果在任意时刻功率超标，输出 "No"。

题解

采用差分的思想，维护一个数组 `arr[]`，`arr[k]` 的含义是对于在时间 k 时，总功率变化 `arr[k]`，所以对于每一个 S_i, T_i, P_i ，我们令

```
arr[S_i] += P_i; arr[T_i] -= P_i;
```

用 `+` 代表开始功能，`-` 代表关闭功能

最后按照从 0 到 2×10^5 ，将 `arr` 进行累加，得到每一个时刻的总功率并跟上限 W 进行比较。

需要注意本题数据超过 `int` 的范围，需要开 `long long`

代码如下：

```
#include<bits/stdc++.h>
```

```

using namespace std;
long long arr[200005];

int main(){
    long long N, W;
    cin >> N >> W;
    for (int i = 1; i <= N; i++) {
        long long x, y, z; cin >> x >> y >> z;
        arr[x] += z; arr[y] -= z; //
    }
    if (arr[0] > W) {
        cout << "No" << endl;
        return 0;
    }
    for (int i = 1; i <= 200000; i++) {
        arr[i] += arr[i - 1]; //累加得到时刻i的值
        if (arr[i] > W) {
            cout << "No" << endl;
            return 0;
        }
    }
    cout << "Yes" << endl;
}

```

I 北洋园的计算器

一、解题思路

1. 问题分析

本题要求计算简单的二元表达式 "a op b =", 其中op是'+', '-'或'*', 计算相应结果。

2. 输入格式解析

输入格式固定为四个部分, 用空格分隔:

1. 整数a
2. 字符op (运算符)
3. 整数b
4. 字符'=' (等号)

需要按顺序读取这四个部分, 但等号不参与计算。

3. 算法设计

这是一个直接的表达式求值问题, 无需复杂算法:

1. **读取输入**: 按照给定格式读取四个值
2. **根据运算符选择运算**:
 - 如果op为'+', 执行加法: result = a + b
 - 如果op为'-', 执行减法: result = a - b
 - 如果op为'*', 执行乘法: result = a × b

3. 输出结果

4. 正确性证明

由于输入格式严格固定，且运算符只有三种可能，算法直接根据运算符执行相应运算，显然正确。

5. 时间复杂度分析

- 读取输入： $O(1)$
- 运算操作： $O(1)$
- 总时间复杂度： $O(1)$

6. 边界情况处理

1. **整数范围**：题目保证a、b和结果都在int范围内，但实际实现时：
 - 在C++中，int通常为32位有符号整数
 - 对于加减法，如果a、b在 $[-10^9, 10^9]$ 范围内，和与差也在 $[-2 \times 10^9, 2 \times 10^9]$ 范围内，不会溢出
 - 对于乘法，最坏情况是 $10^9 \times 10^9 = 10^{18}$ ，可能溢出32位int，但题目明确保证结果在int范围内
2. **负数处理**：标准整数运算已支持负数，无需特殊处理。
3. **运算符验证**：题目保证运算符只能是 '+'、 '-' 或 '*'，无需检查非法运算符。

7. 实现细节

1. **输入读取顺序**：

cpp

```
int a, b;
char op, eq;
cin >> a >> op >> b >> eq;
```

2. **条件判断**：可以使用if-else或switch语句
3. **输出**：直接输出整数结果，无需格式控制

```
#include <bits/stdc++.h>

using namespace std;

int main()
{
    int a, b;
    char op, eq;

    // 读取 a op b =
    cin >> a >> op >> b >> eq;

    int result;
    if (op == '+')
    {
        result = a + b;
```

```
}  
else if (op == '-')  
{  
    result = a - b;  
}  
else if (op == '*')  
{  
    result = a * b;  
}  
cout << result << endl;  
  
return 0;  
  
}
```

J 最爱的古钱币

精简题意：

给定一棵 n 个节点的有根树（根为 1），每个节点有一种颜色。进行 m 次询问，每次给定 (v, k) ，求以 v 为根的子树中，**出现次数** $\geq k$ 的颜色有多少种。 $(n, m \leq 10^5)$

这道题目定位是 中等偏上，是为了卡住一部分有基础的同学的。是真女主(



这个题目用到的思想是 树上启发式合并 (DSU on Tree)，算是稍微小众的知识点，属于进阶算法，不建议零基础的同学直接钻研学习。平时不太多见，但是如果学过 树上启发式合并，这道题目应该是可以容易想到解法的。

今年ICPC 2025成都站 [L. Label Matching](#) 这道题目也可以使用 树上启发式合并 解决。感兴趣的同学在学会本题之后可以尝试一下。

不多介绍树上启发式合并的算法原理，建议学会树链剖分后再进行学习。

树上启发式合并解这个题目的时间复杂度是 $O(n \log n)$ ，造数据的时候卡掉了可能想到的莫队算法。

通过树上启发式合并 (DSU on Tree)，我们可以轻松维护每棵子树内各种钱币的计数，这个就是有关树上启发式合并的最基础的题目类型：[树上数颜色](#)。本题的核心难点在于：**如何利用 $O(1)$ 的时间复杂度，动态统计“出现次数 $\geq k$ 的古钱币有多少种”？**

我们维护两个数组：

- `cnt[i]`：表示第 i 种古钱币当前出现的次数。
- `ans[j]`：表示出现次数 $\geq j$ 的古钱币有多少种。

在 DSU on Tree 的过程中，状态转移逻辑如下：

1. 添加贡献：

当我们遇到一枚古钱币 x 时，它的数量会增加。

- 操作顺序：先将 `cnt[x]` 加 1，由于该颜色的数量达到了一个新的高度 `cnt[x]`，它现在对 `ans[cnt[x]]` 也有了贡献，所以将 `ans[cnt[x]]` 加 1。

2. 移除贡献：

当我们移出一枚古钱币 x 时，它的数量会减少。

- 操作顺序：当前该颜色数量为 `cnt[x]`，减少一枚后它将不再满足 $\geq cnt[x]$ 的条件。因此，先将 `ans[cnt[x]]` 减 1，然后再将 `cnt[x]` 减 1 更新数量。

这样，对于任何一次询问 (v, k) ，我们只需要直接输出 `ans[k]` 即可，查询时间复杂度为 $O(1)$ 。离线处理查询即可。

AC代码

```
#include <bits/stdc++.h>
using namespace std;
const int N = 100005;

int n, m;
int c[N];
vector<int> adj[N];
vector<pair<int, int>> qs[N];

int sz[N], son[N];
int cnt[N];
int ans[N];
int out[N];

int skip_node;

void dfs1(int u, int f) {
    sz[u] = 1;
    for (int v : adj[u]) {
        if (v == f) continue;
        dfs1(v, u);
        sz[u] += sz[v];
        if (sz[v] > sz[son[u]]) son[u] = v;
    }
}

void update(int u, int f, int val) {
    if (val == 1) {
        cnt[c[u]]++;
    }
}
```

```

        ans[cnt[c[u]]]++;
    } else {
        ans[cnt[c[u]]]--;
        cnt[c[u]]--;
    }
    for (int v : adj[u]) {
        if (v == f || v == skip_node) continue;
        update(v, u, val);
    }
}

void dfs2(int u, int f, int keep) {
    for (int v : adj[u]) {
        if (v == f || v == son[u]) continue;
        dfs2(v, u, 0);
    }
    if (son[u]) {
        dfs2(son[u], u, 1);
        skip_node = son[u];
    }

    update(u, f, 1);
    skip_node = 0;

    for (auto &x : qs[u]) {
        out[x.first] = ans[x.second];
    }

    if (!keep) {
        update(u, f, -1);
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    cin >> n >> m;

    for (int i = 1; i <= n; i++) cin >> c[i];

    for (int i = 1; i < n; i++) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    for (int i = 1; i <= m; i++) {
        int u, k;
        cin >> u >> k;
        qs[u].push_back({i, k});
    }

    dfs1(1, 0);
    dfs2(1, 0, 1);
}

```



```

    for (int i = 1; i <= m; i++) {
        cout << out[i] << "\n";
    }

    return 0;
}

```

K 文&理

为了简化证明，将两类人记为A和B

首先，我们观察到一个简单的性质：令 $d[i]$ 表示 i 两边A/B的数量差，那么对于一种人，它的位置的 d 一定是先递减后递增。所以每种动物的排序一定是代价先递增后递减。

结论可以更深入一点。实际上，纵观整个序列，不论AB， d 都是先减后增的。所以呢，我们可以把所有AB一起排序。然后 $f(i,j,k)$ 表示前 i 个人，前缀填了 j 个B， k 个A的最小代价。

其实我们还可以得到更强一点的性质。举 n,m 都为偶数为例：那么前一半位置中，一定有 $n/2$ 个B， $m/2$ 个A。感性地，这不难证明：令B为A，A为B，假如前一半A的数量 $< n/2$ ，那么找到右边的第一个A，把它挪到分界线左边的位置。那么当前位置和目标位置之间全是B，且这些B的 d 都会减小1，且这个A的 d 会减少这些B的数量。

如果 n,m 中有奇数，那么区别不大，只是需要把最大的放在最中间，证明类似。同时我们可以把最大的给剔除，于是就变成了都是偶数的情况。

其实我们观察到另一件重要的事情。在符合我们上述条件的序列中， $\sum d$ 总是一样的。这启发我们其实这里也可以贪心，让 a 更大的值， d 尽量地小。

考虑如下的贪心过程：我们从中心开始往两侧填。假如目前这个是B，且右边能放（即右边数量 $< n/2$ ），那就放右边，代价为右边A的个数乘上 $2a[i]$ ；假如这是A，且左边能放，那就放左边，代价为左边B的数量乘上 $2a[i]$ 。这样能在保证 $\sum d$ 最小的前提下最小化 $\sum d \times a$ 。

代码如下：

```

#include<bits/stdc++.h>
using namespace std;
#define pii pair<int,int>
#define tii tuple<int,int,int>
#define ll long long
#define inf 0x3f3f3f3f
#define ull unsigned long long
#define pll pair<ll,ll>
#define ls d*2
#define rs d*2+1
#define mid (l+r)/2
#define lowbit(x) (x&(-x))
// #define endl "\n"
#define all(x) x.begin(),x.end()
#define int long long
//mt19937 seed;
//uniform_int_distribution<int>num(0,2e9);
const int N=2e6+10;
const int M=110;
int ax[4]={1,-1,0,0},ay[4]={0,0,1,-1};

```

```

int a[1010],b[1010],cnt;
pii c[2010];
void solve(){
    int n,m;
    cin>>n>>m;
    int sa=0,sb=0;
    cnt=0;
    for(int i=1;i<=n;i++)cin>>a[i],sa+=a[i];
    for(int i=1;i<=m;i++)cin>>b[i],sb+=b[i];
    int ans=0;
    if(n&1)ans+=sb;
    if(m&1)ans+=sa;
    sort(a+1,a+1+n);
    sort(b+1,b+1+m);
    for(int i=1;i<=n/2*2;i++)c[++cnt]={a[i],1};
    for(int i=1;i<=m/2*2;i++)c[++cnt]={b[i],2};
    sort(c+1,c+1+cnt);
    int l1=0,l2=0,r1=0,r2=0;
    for(int i=cnt;i;i--){
        if(c[i].second==1){
            if(l1<n/2)ans+=c[i].first*2*l2,l1++;
            else ans+=c[i].first*2*r2,r1++;
        }
        else{
            if(r2<m/2)ans+=c[i].first*2*r1,r2++;
            else ans+=c[i].first*2*l1,l2++;
        }
    }
    cout<<ans<<endl;
}
signed main(){
    ios::sync_with_stdio(false);
    cin.tie(0);cout.tie(0);
    int _=1;cin>>_;
    while(_--){
        solve();
    }
}

```

L 计算几何

题意

在二维坐标系里有 n 个红点和 n 个蓝点，你需要让每个红点都只和一个蓝点相连，最后形成 n 条连线，你需要给出连线方式，保证连线互不相交。题目保证，这 $2n$ 个点不会有坐标一样的点，并且任意三点不共线。

题解

我们最开始一共有 $2n$ 个点，不妨先找到这 $2n$ 个点的凸包。我们随便找到凸包上的一个点 p ，把剩下的点以 p 为坐标原点，按照极角进行排序。

我们按照这个顺序遍历所有的点，假设遍历到点 q ，将 p 点 q 点进行连线，由于三点不共线，可以保证剩下的点都在两侧，并且可以得知两侧红点有多少个点，蓝点有多少个点。

一定存在一个点 q ，使得 q 点与 p 点颜色相反，并且 p 、 q 连线的左侧蓝色点与红色点数目相同，右侧蓝色点与红色点数目也相同，这样的话，把左侧所有点形成一个凸包继续求解，可以保证不会与线段 pq 有交点，右侧同理，因此这个问题可以分治去求。

如果 pq 连线将剩下的点平分的越均匀，越合适，因此选择点的时候可以进行随机选点，其中一种方式是固定选取凸包最左下方的点，但事先要将凸包进行随机旋转

代码如下：

```
#include<bits/stdc++.h>
#define pii pair<int,int>
using namespace std;
const int mod=998244353;
const double eps=1e-8;
struct point{
    double x,y;
    point(){}
    point(double _x,double _y):x(_x),y(_y){}
    point operator + (const point &a) const{
        return point(x+a.x,y+a.y);
    }
    point operator - (const point &a) const{
        return point(x-a.x,y-a.y);
    }
    double operator * (const point &a) const{
        return x*a.y-y*a.x;
    }
    double operator ^ (const point &a) const{
        return x*a.x+y*a.y;
    }
    point operator * (double a){
        return point(a*x,a*y);
    }
};
double jijiao(point O,point A)
{
    return atan2(A.y-O.y,A.x-O.x);
}
point A[20005],B[20005];
int n;
int vis[20005];
int p[20005];
double my=1000000000;
double mx=1000000000;
struct sss{
    point p;
    int f;
    int b;
```

```

};
bool cmp(sss x,sss y)
{
    point O={mx,my};
    return jijiao(O,x.p)<jijiao(O,y.p);
}
mt19937_64 myrand(time(0));
void work(vector<sss> vec)
{
    int xx=myrand();
    int yy=myrand();
    double theta=atan2(xx,yy);
    vector<sss> vc;
    for(auto i:vec)
    {
        point po={i.p.x*cos(theta)-
i.p.y*sin(theta),i.p.x*sin(theta)+i.p.y*cos(theta)};
        vc.push_back({po,i.f,i.b});
    }
    vec=vc;
    vc.clear();
    vector<sss> va,vb;
    my=1000000000;
    mx=1000000000;
    int f;
    for(auto i:vec)
    {
        if(my>i.p.y)
        {
            my=i.p.y;
            mx=i.p.x;
            f=i.f;
        }
        else if(my==i.p.y&&mx>i.p.x)
        {
            my=i.p.y;
            mx=i.p.x;
            f=i.f;
        }
    }
    sss tem;
    for(auto i:vec)
    {
        if(fabs(i.p.x-mx)<eps&&fabs(i.p.y-my)<eps)
        {
            tem=i;
        }
        else
        {
            vc.push_back(i);
        }
    }
    vec=vc;
    vc.clear();
    sort(vec.begin(),vec.end(),cmp);
    int cnt=1;

```

```

sss op;
for(auto i:vec)
{
    if(cnt==0)
    {
        vb.push_back(i);
        continue;
    }
    if(i.f==f) cnt++;
    else cnt--;
    if(cnt!=0)
    {
        va.push_back(i);
    }
    else
    {
        op=i;
    }
}
vec.clear();
if(f==1)
{
    p[tem.b]=op.b;
}
else
{
    p[op.b]=tem.b;
}
if(va.size()) work(va);
va.clear();
if(vb.size()) work(vb);
vb.clear();
}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(0);cout.tie(0);
    cin>>n;
    vector<sss> vec;
    for(int i=1;i<=n;i++)
    {
        double x,y;
        cin>>x>>y;
        A[i]={x,y};
        vec.push_back({A[i],1,i});
    }
    for(int i=1;i<=n;i++)
    {
        double x,y;
        cin>>x>>y;
        B[i]={x,y};
        vec.push_back({B[i],2,i});
    }
    work(vec);
    for(int i=1;i<=n;i++)
    {
        if(i!=1) cout<<' ';
    }
}

```

```
        cout<<p[i];  
    }  
    return 0;  
}
```